ARCHIVOS & PLANTILLAS

Bypass de Filtros y Escalada a RCE

HACKING WEB 2026 // SESIÓN 06

ROADMAP: BLOQUE 1

VULNERABILIDADES DE CARGA DE ARCHIVOS

ENTENDIENDO EL FILE UPLOAD

Funcionalidad Crítica

La capacidad de un usuario para subir archivos es una de las funciones más peligrosas en la web moderna. Transforma datos pasivos en archivos almacenados en el servidor.

El Error del Servidor

Ocurre cuando el servidor no solo guarda el archivo, sino que lo deja en una carpeta donde el motor de ejecución (PHP, ASP, JSP) puede procesarlo.

IMPACTOS CRÍTICOS DEL RCE

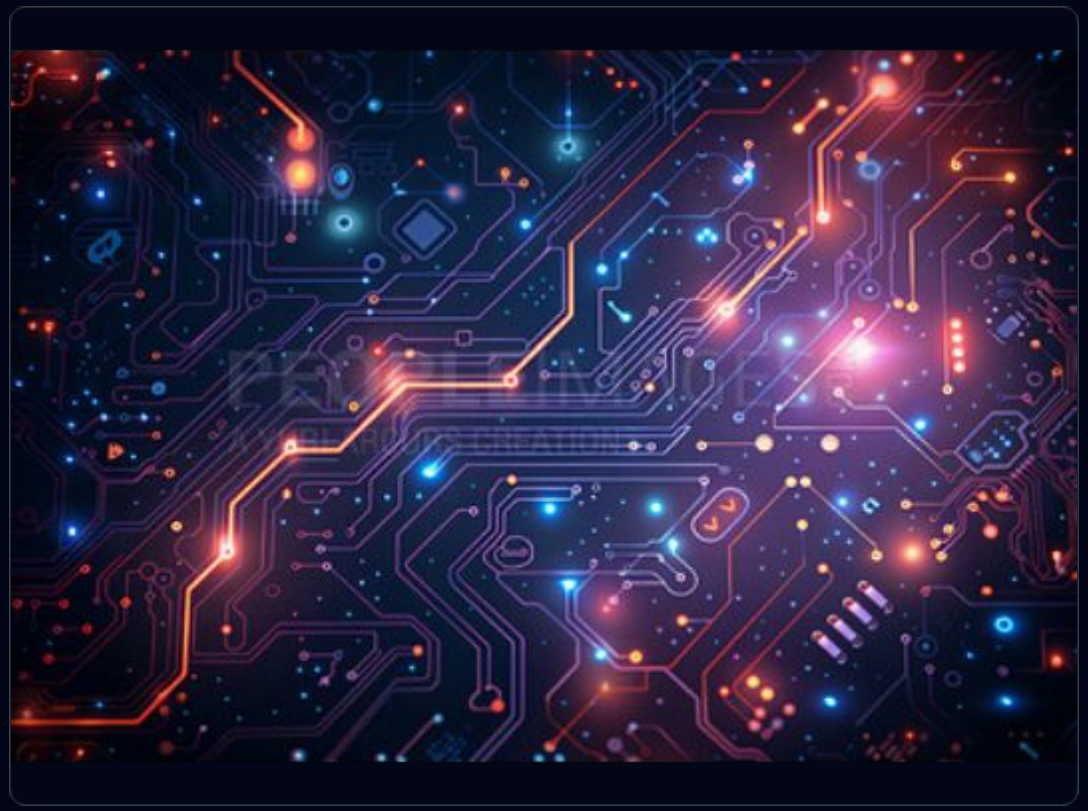
- **Ejecución de Comandos:** Acceso directo al shell del sistema operativo bajo el contexto de la aplicación.
- **Persistencia:** Subida de "Web Shells" para mantener acceso tras la auditoría inicial.
- **Exfiltración:** Lectura de archivos de configuración (.env, config.php) con credenciales de base de datos.
- **Escalada:** Punto de partida para el movimiento lateral dentro de la red corporativa.

ANATOMÍA DE UNA WEB SHELL

Una Web Shell es un script minimalista que recibe comandos vía HTTP y los ejecuta localmente.

```
<?php system($_GET['cmd']); ?>
```

Este payload de una sola línea permite ejecutar cualquier comando en Linux enviando el parámetro ?cmd=id por la URL.



BYPASS DE EXTENSIONES

Lenguaje	Extensión Prohibida	Variantes de Bypass
PHP	.php	.php5, .phtml, .phar, .php.png
ASP.NET	.aspx	.ashx, .asmx, .config
Java	.jsp	.jspx, .jsw, .jsv
Node.js	.js	.mjs, .cjs

FILTRO POR MAGIC BYTES

FFD8

FIRMA JPEG

Firmas de Archivos

Los filtros avanzados validan el contenido real, no solo el nombre. Los primeros bytes de un archivo (Magic Bytes) definen su tipo.

Para saltar este filtro, inyectamos nuestra shell justo después de la firma legítima de una imagen.

CONTENT-TYPE SPOOFING

El servidor confía en la cabecera enviada por el navegador del usuario en el request POST.

```
Content-Type: image/jpeg
```

Simplemente manipulando este valor en Burp Suite, podemos engañar al servidor para que acepte un archivo PHP pensando que es una fotografía JPG.

PATH TRAVERSAL EN UPLOADS

Escape de Directorio

Usar ../../ en el nombre del archivo para sacarlo de la carpeta /uploads.

Ejecución Forzada

Llevar la shell a la raíz del sitio donde la configuración sí permita ejecutar scripts.

Overwrite

Sobreescribir archivos de configuración existentes para inyectar puertas traseras permanentes.

SHELLS POLÍGLOTAS

Inyección en Metadatos

Un archivo políglota es válido para dos formatos a la vez. Mediante exiftool, inyectamos nuestra web shell dentro de los metadatos de una imagen real.

El servidor la valida como imagen, pero el motor de PHP la ejecuta como script.

ROADMAP: BLOQUE 2

SERVER-SIDE TEMPLATE INJECTION (SSTI)

¿QUÉ ES UN TEMPLATE ENGINE?

Diseño vs Datos

Son motores que permiten incrustar lógica de programación (variables, loops) dentro de plantillas estáticas HTML para generar vistas dinámicas.

Motores Comunes

Jinja2 (Python), Mako (Python), Smarty (PHP), Twig (PHP), Thymeleaf (Java), FreeMarker (Java).

CAUSA RAÍZ: INYECCIÓN INSEGURA

El error ocurre cuando el desarrollador comete la imprudencia de concatenar input directo del usuario dentro del string que el motor va a renderizar.

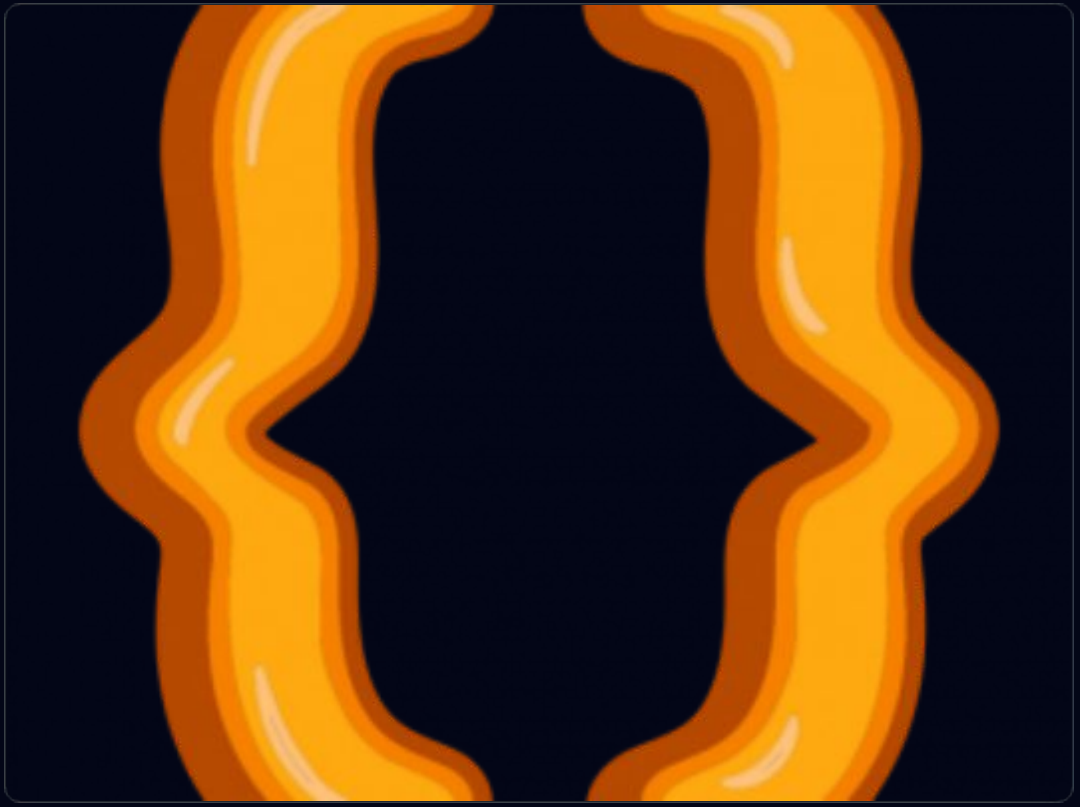
- ✕ **Inseguro:** render_template_string("Hola " + user_input)
- ✓ **Seguro:** render_template("index.html", name=user_input)
- El motor trata el input como código, no como texto plano.

IDENTIFICACIÓN DE MOTORES

Usamos payloads matemáticos para confirmar la vulnerabilidad e identificar el backend.

```
{{7*7}} → 49 (Jinja2 / Twig)
${7*7} → 49 (Smarty / Mako)
```

Si el servidor refleja "49", tenemos una inyección de código confirmada en el motor de renderizado.



NAVEGANDO POR EL MRO

MRO

PYTHON INTROSPECTION

Escalada en Jinja2

En Python, usamos el Method Resolution Order para subir por la cadena de clases hasta llegar a la clase base object.

Desde allí, listamos subclases para encontrar subprocess.Popen y ejecutar comandos de sistema.

SSTI: FUGA DE INFORMACIÓN

Payload	Impacto en el Servidor
{{config}}	Revela todas las variables de entorno de Flask.
{{self.__dict__}}	Expone objetos internos y métodos en memoria.
{{settings.SECRET_KEY}}	Robo de llaves de sesión en Django.
{{_self.env}}	Variables de entorno en Twig (PHP).

BYPASS DE FILTROS EN SSTI

Sin Puntos

Usar la sintaxis de corchetes: config['SECRET_KEY'] en lugar de config.SECRET_KEY.

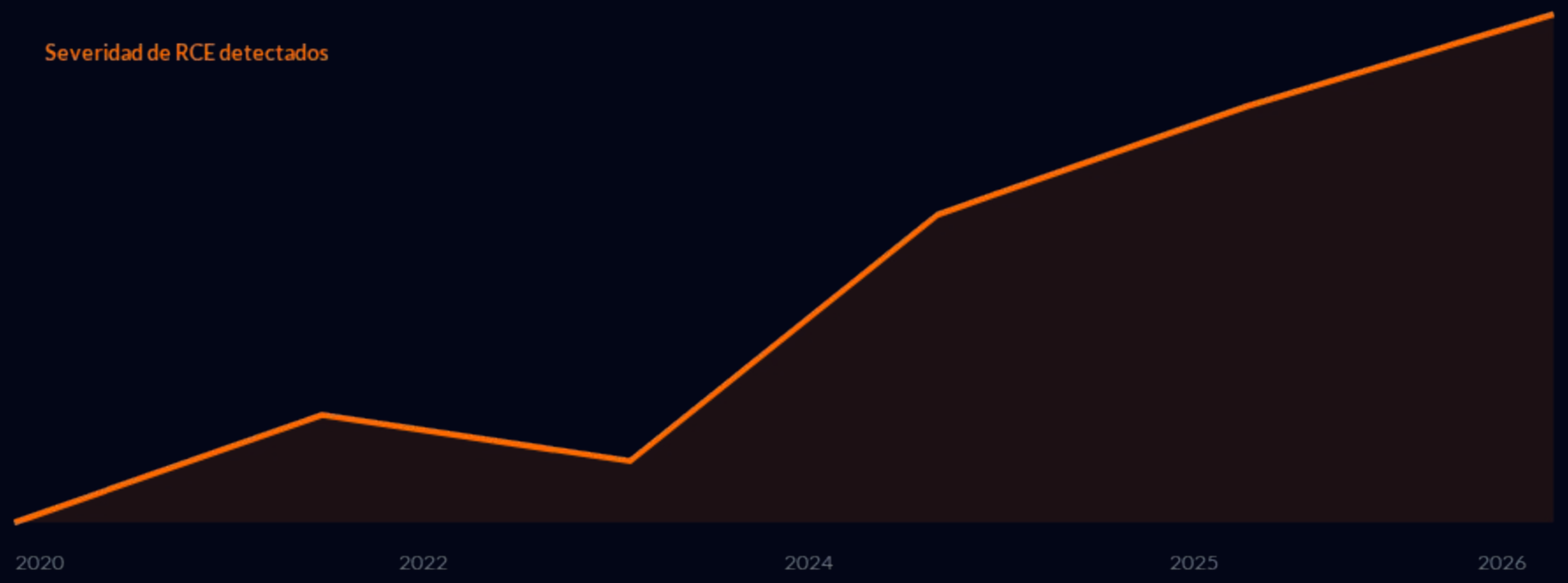
Sin Comillas

Enviar el string por otro parámetro (GET) y llamarlo desde la plantilla: request.args.param.

Filtros

Usar filtros de transformación como |attr para navegar por los atributos del objeto.

TENDENCIAS DE VULNERABILIDADES



A medida que las aplicaciones centralizan más lógica en el servidor, el impacto de SSTI ha crecido exponencialmente.

Cualquier entrada de usuario que el servidor procese como código es una puerta abierta a la toma total del sistema.

— Manual de Auditoría

BLOQUE 3

REMEDIACIÓN Y DEFENSA

DEFENSA EN FILE UPLOAD

- **Whitelisting:** Permitir solo extensiones seguras (jpg, png, pdf) explícitamente.
- **Randomizing:** Renombrar archivos subidos con UUIDs para evitar ejecución de scripts maliciosos.
- **No-Execute:** Configurar la carpeta de subida con permisos de "Solo lectura" en el servidor web.
- **Storage Externo:** Almacenar archivos en buckets externos (AWS S3, GCP) fuera del host de la aplicación.

SECURIZANDO MOTORES

Sandboxing

Configurar el motor de plantillas en entornos restringidos sin acceso a las clases de sistema.

Logic-less

Usar motores como Mustache que no permiten lógica compleja por diseño.

Sanitización

Limpiar y filtrar caracteres de control ({{, }}, $, {) de la entrada del usuario antes del renderizado.

HORA DE LOS LABS

Poniendo en práctica el bypass de firmas y la introspección MRO.

¿PREGUNTAS?

Fin de la Sesión Técnica 06

VANGUARDIUN ACADEMY // HACKING WEB 2026